Layihə təklifi: `std::vector` sıfırdan

Rahim Shikhgayibov · Sərbəst iş · Müasir data strukturları və alqoritmləri (M666aS) · Müəllim: Məmmədov Kamran
github.com/RahimShikhgayibov/cpp-vector-from-scratch

Problem statement

Dinamik massiv proqramlaşdırmada ən çox istifadə olunan məlumat strukturudur: elementləri yaddaşda ardıcıl saxlayır, O(1) indeksləmə verir və lazım olduqca böyüyür. C++-da onu `std::vector` təmin edir. İstifadəsi asandır, amma düzgün implementasiyası obyektin ömrünün əl ilə idarəsini, xam yaddaşı, rule of five-ı, move semantics-i və exception safety-ni tələb edir. Layihədə `std::vector`-un tam işlək analoqu sıfırdan yazılacaq, düzgünlüyü testlərlə sübut olunacaq və xüsusiyyətləri ölçmə ilə tədqiq ediləcək.

Məqsədlər

1. `mystl::vector<T>`: `std::vector`-un bütün interfeysi (custom allocator və `vector<bool>` xaric).
2. Mərhələli təkamül (v1–v5): hər versiya əvvəlkinin konkret problemini həll edir.
3. Düzgünlüyün sübutu: unit testlər, `std::vector` ilə differential test, sanitizer-lər.
4. Aşağıdakı 7 tədqiqat sualına ölçülmüş cavab.

Tədqiqat sualları

1. `push_back` həqiqətən amortized O(1)-dirmi?
2. Böyümə faktoru 2× və 1.5×: hansı yaxşıdır?
3. `reserve` nə qədər qazandırır?
4. Copy ilə move fərqi nədir; `noexcept` unudulanda nə baş verir?
5. Ortaya `insert` / `erase` praktikada nə qədər bahadır?
6. Cache-in təsiri nə qədərdir (vector və linked list traversal)?
7. Nəticə `std::vector`-a nə qədər yaxındır?

Metodologiya və alətlər

Sahə	Seçim
Dil, standart	C++20 (konseptlər, `<=>`, `std::construct_at`)
Build	CMake 3.20+, Ninja; Debug (warning-lər + sanitizer-lər) və Release (`-O2`)
Testlər	doctest; sayğac tipi; exception injection; differential test; coverage (gcov)
Sanitizer-lər	AddressSanitizer + UndefinedBehaviorSanitizer (Linux)
Benchmark	`std::chrono::steady_clock`, ≥ 5 təkrar, median; n = 10^3…10^7; CSV + Python/matplotlib qrafikləri

Əhatə xaricində

Custom allocator dəstəyi (`std::allocator_traits`) və `vector<bool>` ixtisaslaşması.

Gözlənilən nəticələr

- GitHub repository: header-only kitabxana, testlər, benchmark-lar, README.
- Texniki hesabat: dizayn, mürəkkəblik analizi (amortized O(1) isbatı ilə), testlər, benchmark nəticələri.
- Təqdimat və müdafiə.